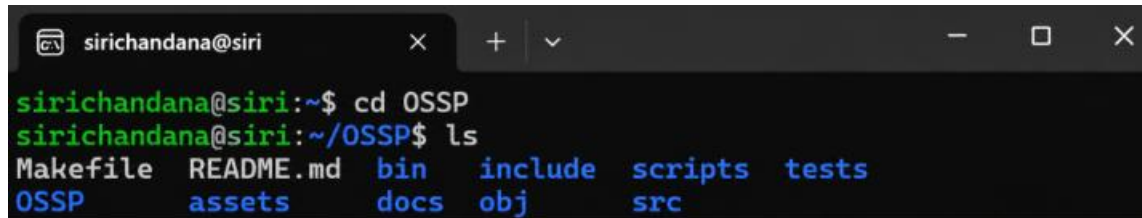


SKILL WEEK-1

Task-1:

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.



```

sirichandana@siri  x  +  v  -  □  x
sirichandana@siri:~$ cd OSSP
sirichandana@siri:~/OSSP$ ls
Makefile  README.md  bin  include  scripts  tests
OSSP      assets    docs  obj      src

```

The image shows a terminal window with a dark background. The window title bar includes a close button (x), a maximize button (square), and a window control button (v). The terminal content shows a user named 'sirichandana' at a host named 'siri'. The user navigates to the directory '~/OSSP' using the 'cd' command. Then, they run the 'ls' command to list the contents of the directory. The output shows a list of files and directories: 'Makefile', 'README.md', 'bin', 'include', 'scripts', 'tests', 'OSSP', 'assets', 'docs', 'obj', and 'src'. The file names are color-coded: 'Makefile' and 'README.md' are green, 'bin', 'include', 'scripts', and 'tests' are blue, and 'OSSP', 'assets', 'docs', 'obj', and 'src' are red.

```
sirichandana@siri: ~  
alled.)  
Preparing to unpack .../tree_2.3.1-1_amd64.deb ...  
Unpacking tree (2.3.1-1) ...  
Setting up tree (2.3.1-1) ...  
Processing triggers for man-db (2.13.1-1build1) ...  
sirichandana@siri:~/OSSP$ tree  
.  
├── Makefile  
├── OSSP  
├── README.md  
├── assets  
│   └── architecture.png  
├── bin  
│   └── my_shell  
├── docs  
│   ├── design.md  
│   └── report.pdf  
├── include  
│   ├── builtin.h  
│   ├── executor.h  
│   ├── parser.h  
│   └── shell.h  
├── obj  
├── scripts  
│   └── run.sh  
├── src  
│   ├── builtin.c  
│   ├── executor.c  
│   ├── main.c  
│   ├── parser.c  
│   └── shell.c  
└── tests  
    ├── test_executor.c  
    └── tests_parser.c  
  
10 directories, 18 files  
sirichandana@siri:~/OSSP$ |
```

Task-2:

To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.

task-1: Using a manual command understand the fork() and exec() system calls

task-2: Write a C program to create a new process and accepting the user commands using fork() and exec() system calls.

Example:

```
#include <stdio.h> #include <unistd.h>
```

```
int main() {    pid_t pid;    pid = fork();
```

```

// Create a new process    if (pid < 0) {
printf("Fork failed!\n");

    }

    else if (pid == 0) {    // Child
Process        printf("This is the Child
Process.\n");        printf("Child PID =
%d\n", getpid());

    }

else {

    // Parent Process        printf("This is
the Parent Process.\n");
printf("Parent PID = %d\n", getpid());
printf("Child PID = %d\n", pid);

    }

    return 0;
}

```

```

sirichandana@siri:~$ ps -f
UID          PID    PPID  C  STIME TTY          TIME CMD
arikela      350     349   0  05:19 pts/0        00:00:00 -bash
arikela      714     350  99  05:39 pts/0        00:00:00 ps -f
sirichandana@siri:~$ echo $$
350
sirichandana@siri:~$ pstree -p
systemd(1)─agetty(219)
            └─chronyd-starter(154)─chronyd(258)─chronyd(264)
            └─cron(155)
            └─dbus-daemon(156)
            └─init-systemd(Ub(2)─SessionLeader(348)─Relay(35+
                  │             │   └─init(7)─{init}(8)
                  │             │   └─login(351)─bash(434)
                  │             └─{init-systemd(Ub3(9)
            └─networkd-dispat(162)
            └─rsyslogd(187)─{rsyslogd}(216)
                  │       └─{rsyslogd}(217)
                  └─{rsyslogd}(218)
            └─systemd(400)─(sd-pam)(402)
            └─systemd-journal(50)
            └─systemd-logind(174)
            └─systemd-resolve(85)
            └─systemd-udevd(91)
            └─unattended-upgr(238)─{unattended-upgr}(304)
sirichandana@siri:~$ man 2 fork

```

NAME

fork - create a child process

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

```
#include <unistd.h>
```

```
pid_t fork(void);
```

DESCRIPTION

fork() creates a new process by duplicating the calling process. The new process is referred to as the child process. The calling process is referred to as the parent process.

The child process and the parent process run in separate memory spaces. At the time of fork() both memory spaces have the same content. Memory writes, file mappings (mmap(2)), and unmappings (munmap(2)) performed by one of the processes do not affect the other.

The child process is an exact duplicate of the parent process except for the following points:

- The child has its own unique process ID, and this PID does not match the ID of any existing process group (setpgid(2)) or session.
- The child's parent process ID is the same as the parent's process ID.
- The child does not inherit its parent's memory locks

nual page fork(2) line 1/248 13% (press h for help or q to quit)

```
sirichandana@siri:~$ ps -f
UID          PID    PPID    C   STIME TTY          TIME CMD
arikela      350     349    0   05:19 pts/0        00:00:00 -bash
arikela      714     350    0   05:52 pts/0        00:00:00 sleep 30
arikela      885     350    0   05:52 pts/0        00:00:00 ps -f
sirichandana@siri:~$
```

NAME

execl, execlp, execl, execv, execvp, execvpe - execute a file

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

```
#include <unistd.h>
```

```
extern char **environ;
```

```
int execl(const char *path, const char *arg, ...  
          /*, (char *) NULL */);
```

```
int execlp(const char *file, const char *arg, ...  
          /*, (char *) NULL */);
```

```
int execl(const char *path, const char *arg, ...  
          /*, (char *) NULL, char *const envp[] */)
```

```
;
```

```
int execv(const char *path, char *const argv[]);
```

```
int execvp(const char *file, char *const argv[]);
```

```
int execvpe(const char *file, char *const argv[], char *c  
onst envp[]);
```

Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

```
execvpe():  
    _GNU_SOURCE
```

DESCRIPTION

The `exec()` family of functions replaces the current process image with a new process image. The functions described in this manual page are layered on top of ex-

Manual page `exec(3)` line 1 (press h for help or q to quit)

```

sirichandana@siri:~$ pstree -p
systemd(1)─agetty(219)
          └─chronyd-starter(154)─chronyd(258)─chronyd(264)
          └─cron(155)
          └─dbus-daemon(156)
          └─init-systemd(Ub(2))─SessionLeader(348)─Relay(35+
                                └─init(7)─{init}(8)
                                └─login(351)─bash(434)
                                └─{init-systemd(Ub)}(9)
          └─networkd-dispat(162)
          └─rsyslogd(187)─{rsyslogd}(216)
                        └─{rsyslogd}(217)
                        └─{rsyslogd}(218)
          └─systemd(400)─(sd-pam)(402)
          └─systemd-journal(50)
          └─systemd-logind(174)
          └─systemd-resolve(85)
          └─systemd-udev(91)
          └─unattended-upgr(238)─{unattended-upgr}(304)
          └─wsl-pro-service(831)─{wsl-pro-service}(832)
                                └─{wsl-pro-service}(833)
                                └─{wsl-pro-service}(834)
                                └─{wsl-pro-service}(835)
                                └─{wsl-pro-service}(836)
                                └─{wsl-pro-service}(837)
                                └─{wsl-pro-service}(840)

sirichandana@siri:~$ bash -c 'echo Before;exec ls;echo aft
er'
Before
OSSP OS_LAB filename filename.c namedirectory
after

```

Task-2

```
sirichandana@siri:~$ gcc task2.c -o task2
sirichandana@siri:~$ ./task2
```

```
Parent Process
Parent PID =1425
Child PID =1426
Child Process
Child PID= 1426
```

```
GNU nano 8.7.1 task2.c
#include <stdio.h>
#include <unistd.h>
int main(){
pid_t pid;
pid=fork();

if(pid<0){
printf("Fork failed!\n");
}
else if(pid==0){
printf("Child Process\n");
printf("Child PID= %d\n",getpid());
}
else{
printf("Parent Process\n");
printf("Parent PID =%d\n",getpid());
printf("Child PID =%d\n",pid);
}
return 0;
}
```

[Read 20 lines]

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
sirichandana@siri:~$ gcc exec_demo.c -o exec_demo
sirichandana@siri:~$ ./exec_demo
Before exec
OSSP      exec_demo.c  filename      task2
OS_LAB    exec_demo    filename.c    task2.c
exec_demo exec_demo.c  namedirectory
sirichandana@siri:~$
```

```
GNU nano 8.7.1      ecec_demo.c
#include <stdio.h>
#include <unistd.h>
int main(){
printf("Before exec\n");
char *args[]={ "ls", NULL };
execvp(args[0],args);
printf("After ecec\n");
return 0;
}|

[ Wrote 9 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(){
pid_t pid;
pid=fork();

if(pid<0){
printf("Fork Failed!\n");
}
else if(pid==0){
printf("Child Process\n");
char *args[]={"ls",NULL};
execvp(args[0],args);
//executes if execp fails
printf("exec failed!\n");
}
else{
printf("Parent Process\n");
wait(NULL);
printf("Child finished\n");
}
return 0;
}
```

[Read 25 lines]

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^_ Replace

^K Cut

^U Paste

^T Execute

^J Justify

```
sirichandana@siri:~$ nano fork_exec.c
sirichandana@siri:~$ gcc fork_exec.c -o fork_exec
sirichandana@siri:~$ ./fork_exec
Parent Process
Child Process
OSSP      ecec_demo.c      filename      fork_exec.c      task2.c
OS_LAB    exec_demo      filename.c    namedirectory
ecec_demo exec_demo.c    fork_exec    task2
Child finished
```

GNU nano 8.7.1

task2.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(){
char command[100];
printf("Enter command: ");
scanf("%s",command);
pid_t pid=fork();

if(pid<0){
printf("Fork failed!\n");
}
else if(pid==0){
printf("Child Process\n");
char *args[]={command,NULL};
execvp(args[0],args);
printf("Command execution failed!\n");
}
else{
printf("Parent Process\n");
printf("Parent PID =%d\n",getpid());
printf("Child PID =%d\n",pid);
wait(NULL);
printf("Child process completed.\n");
}
return 0;
}
```

[Read 27 lines]

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

sirichandana@siri:~\$ nano task2.c

sirichandana@siri:~\$ gcc task2.c -o task2

sirichandana@siri:~\$./task2

Enter command: ls

Parent Process

Parent PID =2200

Child PID =2203

Child Process

OSSP ecec_demo.c

filename

fork_exec.c

task2.c

OS_LAB exec_demo

filename.c

namedirectory

ecec_demo exec_demo.c

fork_exec

task2

Child process completed.